

Bot de *Market Making* Adaptativo

XRPUSDC — Binance Futures (USDS-M)

Documentación técnica de la estrategia v1.0

Proyecto *alpha-driven*

10 de agosto de 2026

Resumen ejecutivo

Este documento describe la estrategia de *market making* adaptativo implementada en el repositorio **alpha-driven**: un bot que cotiza simultáneamente un precio de compra (bid) y uno de venta (ask) alrededor del precio medio del libro de órdenes del par XRPUSDC en Binance Futures (USDS-M), con tamaño y *spread* ajustados por la volatilidad observada y el inventario acumulado. El objetivo es capturar el *spread* como ingreso neto de comisiones, *funding* y deslizamiento ($\text{NetPnL} > 0$, véase §6).

La implementación cumple las reglas no negociables de la Sección 0 de la especificación (§1): test-net primero, presupuestos de riesgo definidos como constantes, kill switch probado explícitamente y detención ante situaciones irreconciliables. La decisión de diseño aprobada habilita la corrida integrada *dry-run* en Nivel de exposición 0 sin enviar órdenes reales. A la fecha de este documento la estrategia operó su primera corrida en mainnet (autorización humana explícita, §0.2) con resultados de reconciliación exitosos; los presupuestos de riesgo siguen marcados como propuestas pendientes de confirmación (§0.4).

Tablero de estado (al 10/08/2026)

Componente	Estado
Risk Engine (tests unitarios)	13/13 OK
Execution Engine (tests unitarios)	14/14 OK
Alpha Model (tests unitarios)	18/18 OK
Kill switch (prueba explícita)	4/4 OK
Primera corrida mainnet	Operada: 8 órdenes reales, 4.9 XRP, exit 0
Presupuestos de riesgo	Propuestos, pendientes de confirmación (§0.4)

Índice

Resumen ejecutivo	1
1. Objetivo y reglas no negociables (§0)	2
1.1. Objetivo	2

1.2. Reglas no negociables (§0)	2
1.3. Decisión de diseño aprobada: Nivel 0 / dry-run	2
2. Arquitectura	2
2.1. Interfaz única con Binance	2
2.2. WebSocket para datos en tiempo real	3
2.3. Flujo de datos	3
2.4. Instrumento	3
3. Estrategia de cotización	3
3.1. Régimen de volatilidad	3
3.2. Precio medio y spread objetivo	4
3.3. Precios de cotización (bid/ask) con skew	4
3.4. Tamaños de orden y clamp	4
4. Risk Engine	5
4.1. Rol	5
4.2. Presupuestos como desigualdades	5
4.3. Niveles de exposición	5
4.4. Kill switch (§13)	6
5. Ejecución y reconciliación	6
5.1. Órdenes maker	6
5.2. Identificación idempotente de órdenes	6
5.3. Detección de fills y reconciliación	6
5.4. Concurrencia y rate-limit	7
6. Métricas de rendimiento	7
6.1. NetPnL	7
6.2. Una operación ganadora NO es evidencia (§19)	7
7. Operaciones	7
7.1. Watchdog	7
7.2. Pre-flight en mainnet	8
7.3. Aislamiento en la VPS	8
A. Glosario	8
B. Constantes clave de configuración	8

1. Objetivo y reglas no negociables (§0)

1.1. Objetivo

El sistema debe generar ganancia neta consistente operando como *market maker* en el par XRPUSDC de Binance Futures, adaptando su comportamiento a la volatilidad del mercado y al inventario acumulado, sin superar nunca los límites de riesgo definidos en `config.py`.

1.2. Reglas no negociables (§0)

Las siguientes reglas tienen prioridad sobre el resto de la especificación y sobre cualquier decisión operativa:

1. **Testnet primero.** Mainnet SOLO con autorización humana explícita. No se llama `init_client(real=True)` ni `set_testnet(False)` sin esa autorización (§0.1).
2. **Detenerse y preguntar** ante: primera orden en mainnet, subir el Nivel de exposición, kill switch activado, falta de una función de API (no escribir implementación paralela), posición no reconciliable o credenciales nuevas (§0.2).
3. **Presupuesto de riesgo** como constantes en `config.py`, nunca números improvisados (§0.4):

`MAX_DAILY_LOSS_USDC      MAX_POSITION_NOTIONAL_USDC      MAX_DRAWDOWN_PCT      MAX_LEVERAGE_USE`

Si no están definidos: proponer valores conservadores, marcarlos “pendiente de confirmación” y no operar con fondos reales.
4. **Credenciales** (§0.5): nunca loguearlas, imprimirlas, commitearlas ni rotarlas sin preguntar. `.env` está en `.gitignore`.
5. **Commits chicos y frecuentes** (§0.3) y `STATUS.md` actualizado cada 15–20 minutos.

1.3. Decisión de diseño aprobada: Nivel 0 / dry-run

Cuando `EXPOSURE_LEVEL = 0` el bot NO queda bloqueado en tamaño cero: puede simular órdenes usando la constante de simulación `SIMULATION_QUOTE_MULTIPLIER` de `config.py` en lugar del multiplicador 0.0 del nivel 0. Esto habilita la corrida integrada *dry-run*. Los niveles reales (≥ 1) NO se modifican y subirlos sigue requiriendo autorización humana (§0.2). En Nivel 0 jamás se envían órdenes reales (§0.1/§21).

2. Arquitectura

2.1. Interfaz única con Binance

`API_binance_futuros.py` es la única interfaz con Binance Futures (§1). No se duplican funciones que ya existen; ante una función faltante se debe detener y preguntar (§0.2). Sus características:

- Inicialización: `init_client(real: bool = False)`. Testnet usa el modo nativo de `python-binance`; mainnet fuerza URLs reales.
- Los métodos devuelven `BinanceAPIException` como resultado en vez de lanzarla: el llamador debe chequear el tipo de retorno.

- `_call_with_retry(func, retries=2, delay=0.5, ...)` envuelve las llamadas críticas.
- Sincronización de tiempo: `adjust_client_time()` más un hilo daemon cada 300 s que fija `client.timestamp_offset`.

2.2. WebSocket para datos en tiempo real

El flujo de datos es por WebSocket; REST se usa solo para configuración, estado, órdenes, recuperación y reconciliación (§2). URLs: real `wss://fstream.binance.com/public/ws`, testnet `wss://stream.binancefuture.com/public/ws`. Reintento cada 5 s con `ping_interval=20`. Si se superan los reintentos, se activa el kill switch (§13).

Los módulos expuestos y sus callbacks:

- `websocket_bookticker.py` → `BookTickerWebSocket(symbol, on_bookticker, real=False)`, `stream {symbol}@bookTicker`. Callback: `on_bookticker(bid_price, bid_qty, ask_price, ask_qty, timestamp)`.
- `websocket_depth.py` → `DepthWebSocket(symbol, on_depth, real=False, depth_levels=5)`, `stream {symbol}@depth{levels}@100ms`. Callback: `on_depth(bids, asks, depth, timestamp)` con `bids` y `asks` como listas [precio, qty].
- `websocket_trades.py` → `TradeWebSocket(symbol, on_trade, real=False)`, `stream {symbol}@trade`. Callback: `on_trade(price, qty, is_buyer_maker, timestamp)`.

2.3. Flujo de datos

El estado de mercado (`market_state.py`) consume los tres streams, mantiene libro, últimos precios y régimen de volatilidad con `threading.Lock`. El `market_maker.py` es el orquestador: cada ciclo calcula cotizaciones, consulta el Risk Engine y delega el envío al `execution_engine.py`. El `inventory_manager.py` lleva el inventario y el PnL realizado.

Capas	Módulos
Datos	<code>websocket_bookticker/depth/trades.py</code> , <code>market_state.py</code>
Lógica	<code>alpha_model.py</code> (cotizaciones), <code>inventory_manager.py</code>
Riesgo	<code>risk_engine.py</code> (rechazos y kill switch)
Ejecución	<code>execution_engine.py</code>
Orquestación	<code>market_maker.py</code> , <code>run_dry_run.py</code> , <code>run_mainnet.py</code>
Configuración	<code>config.py</code>

2.4. Instrumento

El instrumento es `XRPUSDC`, con el símbolo como parámetro de configuración (no hardcodeado). Las specs reales del símbolo (precisiones, tick/step size, min quantity/notional) se consultan dinámicamente — nunca se asumen (§3).

3. Estrategia de cotización

3.1. Régimen de volatilidad

La volatilidad se estima como el desvío estándar muestral de los retornos logarítmicos del precio medio del libro:

$$r_t = \ln\left(\frac{P_t^{\text{mid}}}{P_{t-1}^{\text{mid}}}\right), \quad \hat{\sigma} = \sqrt{\frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2} \quad (1)$$

donde n es el número de retornos en la ventana (`VOLATILITY_WINDOW_SEC`) y cada retorno corresponde al intervalo de muestreo `SAMPLING_INTERVAL_SEC` (5 s). La volatilidad NO se anualiza (§0.6). Como la ventana puede tener huecos, el régimen se reescala a un intervalo de referencia `INTERVALS_PER_MINUTE = 12`:

$$\hat{\sigma}_{\text{mín}} = \hat{\sigma} \cdot \sqrt{12}, \quad \hat{\sigma}_{\text{ref}} = \hat{\sigma} \cdot \sqrt{\frac{\text{SAMPLING_INTERVAL_SEC}}{\text{intervalo medio observado}}} \quad (2)$$

Convención crítica (§0.6): cada factor $\sqrt{T}$ se aplica exactamente una vez. La volatilidad *efectiva* que escala las distancias de cotización es:

$$\sigma_{\text{efectiva}} = \hat{\sigma}_{\text{mín}} \cdot \sqrt{\text{CYCLE_INTERVAL_SEC}} = \hat{\sigma} \cdot \sqrt{12} \cdot \sqrt{\text{CYCLE_INTERVAL_SEC}} \quad (3)$$

Aplicar $\sqrt{T}$ dos veces (volatilidad anualizada y luego reescalada por el ciclo) es un error conocido del proyecto (§0.6) y está cubierto por tests del alpha model.

3.2. Precio medio y spread objetivo

El precio medio del libro (mid) se calcula con el mejor bid y el mejor ask:

$$P_{\text{mid}} = \frac{P_{\text{bid}} + P_{\text{ask}}}{2} \quad (4)$$

El *spread* objetivo es proporcional a la volatilidad efectiva:

$$S = k \cdot \sigma_{\text{efectiva}} \quad (5)$$

donde k es un factor de escala de `config.py`. A mayor volatilidad, mayor spread para compensar el riesgo de inventario.

3.3. Precios de cotización (bid/ask) con skew

El modelo ajusta los precios alrededor del mid con un sesgo (*skew*) dependiente del inventario: si se tiene inventario largo (comprado), la cotización de venta se acerca al mid (incentivo a desacumular) y la de compra se aleja; simétricamente para inventario corto:

$$P_{\text{bid}} = P_{\text{mid}} - \frac{S}{2} + \delta_{\text{inv}}, \quad P_{\text{ask}} = P_{\text{mid}} + \frac{S}{2} + \delta_{\text{inv}} \quad (6)$$

con el sesgo δ_{inv} calculado por `alpha_model.py` en función del inventario normalizado (convención de signo: inventario > 0 es LARGO, < 0 es CORTO; en el libro, BUY suma cantidad e SELL resta). Los precios se redondean al *tick size* del símbolo consultado dinámicamente (§3).

3.4. Tamaños de orden y clamp

El tamaño base de cada lado es `BASE_ORDER_SIZE_XRP`. Cuando ya existe inventario, el tamaño del lado que reduce inventario (*reduce*) se calcula como:

$$q_{\text{reduce}} = \min(\text{MAX_ORDER_SIZE_XRP}, |\text{inventario}| + \text{BASE_ORDER_SIZE_XRP}) \quad (7)$$

El clamp contra `MAX_ORDER_SIZE_XRP` es necesario porque una orden de reducción no debe superar el tamaño máximo de orden del símbolo; durante la primera corrida mainnet se observó el error de Binance `order_size_exceeded` con inventario de 4.9 XRP, que bloqueaba la reducción de posición. Con el clamp, el lado reduce queda acotado a `MAX_ORDER_SIZE_XRP` y la posición se puede desacumular sin error (fix 632f9c3).

El tamaño total expuesto por lado queda limitado por el Risk Engine (§4); en Nivel 0 el multiplicador de cotización es `SIMULATION_QUOTE_MULTIPLIER` (decisión de diseño aprobada, §1.3).

4. Risk Engine

4.1. Rol

El Risk Engine (`risk_engine.py`) es un módulo independiente que lee los límites de `config.py` y decide si una operación es admisible. Toda orden propuesta pasa por él antes de enviarse; ante un rechazo (`allowed=False`) la orden no se envía. Debe ser probado con tests unitarios que verifiquen los rechazos (§12) — actualmente 13/13 OK.

4.2. Presupuestos como desigualdades

Los límites son constantes en `config.py` (§0.4), propuestas conservadoras pendientes de confirmación humana:

Límite	Desigualdad
Pérdida diaria	pérdida del día $\leq \text{MAX_DAILY_LOSS_USDC}$
Nocional de posición	$ q \cdot P \leq \text{MAX_POSITION_NOTIONAL_USDC}$
Drawdown	drawdown $\leq \text{MAX_DRAWDOWN_PCT}$
Apalancamiento	apalancamiento $\leq \text{MAX_LEVERAGE_USED}$
Tamaño de orden	$q \leq \text{MAX_ORDER_SIZE_XRP}$

Si no estuvieran definidos, el bot no operaría con fondos reales.

4.3. Niveles de exposición

El multiplicador de tamaño se controla con `EXPOSURE_LEVEL` $\in \{0, 1, 2, 3\}$:

Nivel	Modo	Comportamiento
0	Dry-run	Simulación en memoria con <code>SIMULATION_QUOTE_MULTIPLIER</code> . Jamás envía órdenes.
1	Testnet real	Tamaño reducido; operación real solo en testnet.
2	Testnet completo	Tamaño nominal con los límites de config.
3	Mainnet	Operación real con fondos. Exige autorización humana explícita (§0.1/§0.2).

Subir de nivel de exposición real requiere autorización humana (§0.2).

4.4. Kill switch (§13)

El kill switch se activa ante cualquiera de estas condiciones: pérdida excesiva, comportamiento anómalo, WebSocket desconectado, estado inconsistente, órdenes desconocidas, posición no reconciliada, error de API repetitivo, precio anómalo, pérdida de sincronización o exposición inesperada.

Procedimiento de emergencia:

1. Cancelar TODAS las órdenes abiertas (*cancel all open orders*).
2. Reconciliar la posición real contra el estado interno.
3. Nunca asumir que cancelar órdenes implica no tener posición: el inventario se verifica contra la cuenta.

El kill switch debe probarse explícitamente antes de confiar en él (§13) — cubierto por 4 tests OK. En Nivel 0, su activación es análoga pero no toca la cuenta real.

5. Ejecución y reconciliación

5.1. Órdenes maker

Todas las órdenes son *maker* (post-only) para capturar el spread sin pagar comisiones de taker:

- `timeInForce='GTX'` (post-only) en cada orden.
- `positionSide='BOTH'` (modo one-way).
- `reduceOnly=True` cuando corresponde (lado que reduce inventario).
- Tamaño y precio validados contra *min quantity*, *min notional*, *tick size* y `PERCENT_PRICE` del símbolo.

La verificación de que el precio cotizado es realmente *maker* (no cruza el libro, §10) se hace antes de cada envío.

5.2. Identificación idempotente de órdenes

Cada orden lleva un `client_order_id` único por lado:

MM-<lado>-<timestamp>-<contador> # ej.: MM-BUY-1723300000123-1

Los IDs de bid y ask son independientes y se purgan al confirmar la muerte de la orden (§0.6: compartir IDs bid/ask sin purgar genera “órdenes fantasma” acumuladas). El contador garantiza idempotencia ante reinicios: al reenviar, el mismo ID no duplica la orden.

5.3. Detección de fills y reconciliación

La detección de *fills* ocurre en el *refresh* de órdenes abiertas: si una orden desaparece del libro (vía `sync_removed`) y la posición de la cuenta creció en el mismo sentido, se registra un *fill* real y se actualiza el inventario (en modo *dry-run*, el fill es simulado en memoria).

La reconciliación distingue dos casos que antes se confundían:

Caso	Tratamiento
Posición vacía (0)	Posición legítima; pre-flight continúa
Error de API (<code>None</code>)	Falla de consulta; detenerse y reintentar

Este fix (345c7a7) destrabó el pre-flight de mainnet con cuenta limpia. Ante una posición que no se puede reconciliar, la regla §0.2 ordena detenerse y preguntar.

5.4. Concurrencia y rate-limit

El `execution_engine.py` protege el estado compartido con un `threading.Lock` y no sincroniza estado/inventario en cada ciclo sin control de frecuencia (rate-limit, §0.6). Los envíos críticos pasan por `_call_with_retry`.

6. Métricas de rendimiento

6.1. NetPnL

El criterio de éxito es ganancia neta:

$$\text{NetPnL} = \text{GrossPnL} - \text{fees} - \text{funding} - \text{slippage} \quad (8)$$

(§18 de la especificación). Los componentes:

- **GrossPnL**: diferencia de precios de las operaciones cerradas, con PnL realizado por media móvil en `inventory_manager.py`.
- **Fees**: comisiones de *maker/taker* acumuladas por separado (`total_fees`) y neteadas aquí.
- **Funding**: tasa por 8 h tomada de `FUNDING_RATE_PER_8H` de `config.py` (el bot no consulta `get_funding_rate`).
- **Slippage**: deslizamiento estimado entre el precio cotizado y el precio de ejecución.

6.2. Una operación ganadora NO es evidencia (§19)

La rentabilidad se evalúa con $\text{NetPnL} > 0$ acumulado, no con operaciones individuales. Una operación ganadora puede ser ruido o compensar pérdidas previas; solo el PnL neto agregado valida la estrategia.

7. Operaciones

7.1. Watchdog

El bot corre un watchdog que vigila la actividad del ciclo de cotización. El timeout depende del número de ciclos configurado (`-cycles`) con un piso:

$$W_{\text{watchdog}} = \text{máx}(125 \text{ s}, 6 \cdot \text{cycles} + 30 \text{ s}) \quad (9)$$

Si el ciclo no reporta actividad dentro de W_{watchdog} , el proceso termina con error (y, en niveles reales, se activa el kill switch si corresponde).

7.2. Pre-flight en mainnet

Antes de operar, `run_mainnet.py` valida condiciones y credenciales y tolera una posición pre-existente como *round trip* si se mantiene dentro de los límites:

$$|q_{\text{pos}} \cdot P_{\text{mark}}| \leq \text{MAX_POSITION_NOTIONAL_USDC} \quad \text{y} \quad |q_{\text{pos}}| \leq 2 \cdot \text{BASE_ORDER_SIZE_XRP} \quad (10)$$

Fuera de esos límites la posición se considera no reconciliable y el bot se detiene y pregunta (§0.2). Además: valida `PERCENT_PRICE` y `requiredMarginPercent`, consulta el *mark price*, fija `set_leverage(MAX_LEVERAGE_USED)` y exige confirmación interactiva (`CONFIRMAR`); la bandera `-yes` solo se usa bajo supervisión directa o CI. Operar exige `config.REAL=True`, `EXPOSURE_LEVEL ≥ 1` y `BUDGETS_CONFIRMED=True`.

7.3. Aislamiento en la VPS

En la misma VPS corren otros bots en producción (grid bot, bot A-S en BTC/USDC). El proceso nuevo corre aislado: venv propio, logs propios, sin compartir puertos, archivos ni el límite de rate-limit (§0.3). No tocar ni reiniciar los procesos de los otros bots.

A. Glosario

Ask / Bid Mejor precio de venta / de compra del libro.

Futures USDS-M Contratos de futuros liquidados en stablecoin (USDC en este caso).

GTX *Good 'til cancelled, post-only*: orden maker que se cancela si ejecutaría como taker.

Inventario Cantidad de XRP acumulada por el bot; positiva = larga, negativa = corta.

Kill switch Mecanismo de emergencia que cancela todas las órdenes y reconcilia la posición ante condiciones anómalas (§13).

Largo / Corto Posición de compra / de venta.

Mark price Precio de referencia usado para liquidaciones y cálculo de notional.

Post-only Ver *GTX*.

Round trip Posición pre-existente considerada válida si cumple los límites de pre-flight (§7.2).

Skew Sesgo aplicado a los precios de cotización según inventario (§3.3).

Testnet Red de pruebas de Binance con fondos simulados.

Tick size / Step size Incremento mínimo de precio / de cantidad de un símbolo.

B. Constantes clave de configuración

Constante	Valor	Rol
SYMBOL	"XRPUSDC"	Instrumento (parámetro, no hardcodeado)
REAL	True	Mainnet autorizada 10/08/2026 (§0.2)

Constante	Valor	Rol
EXPOSURE_LEVEL	0–3	Nivel de exposición (tabla §4.3)
SAMPLING_INTERVAL_SEC	5.0	Intervalo de muestreo de retornos
INTERVALS_PER_MINUTE	12	Reescala de volatilidad (§3.1)
VOLATILITY_WINDOW_SEC	—	Ventana de estimación de vol
CYCLE_INTERVAL_SEC	—	Duración del ciclo de cotización
MAX_DAILY_LOSS_USDC	10.0	Límite diario de pérdida (§0.4)
MAX_POSITION_NOTIONAL_USDC	—	Límite de nocional
MAX_DRAWDOWN_PCT	—	Límite de drawdown
MAX_LEVERAGE_USED	—	Apalancamiento máximo
BASE_ORDER_SIZE_XRP	—	Tamaño base por lado
MAX_ORDER_SIZE_XRP	—	Tamaño máximo de orden (clamp §3.4)
SIMULATION_QUOTE_MULTIPLIER	—	Multiplicador de simulación (Nivel 0)
FUNDING_RATE_PER_8H	—	Tasa de funding usada en NetPnL

Nota: los valores marcados con “—” corresponden a la configuración vigente en `strategy/config.py`; consultar ese archivo como fuente de verdad antes de modificar cualquier límite (§0.4).